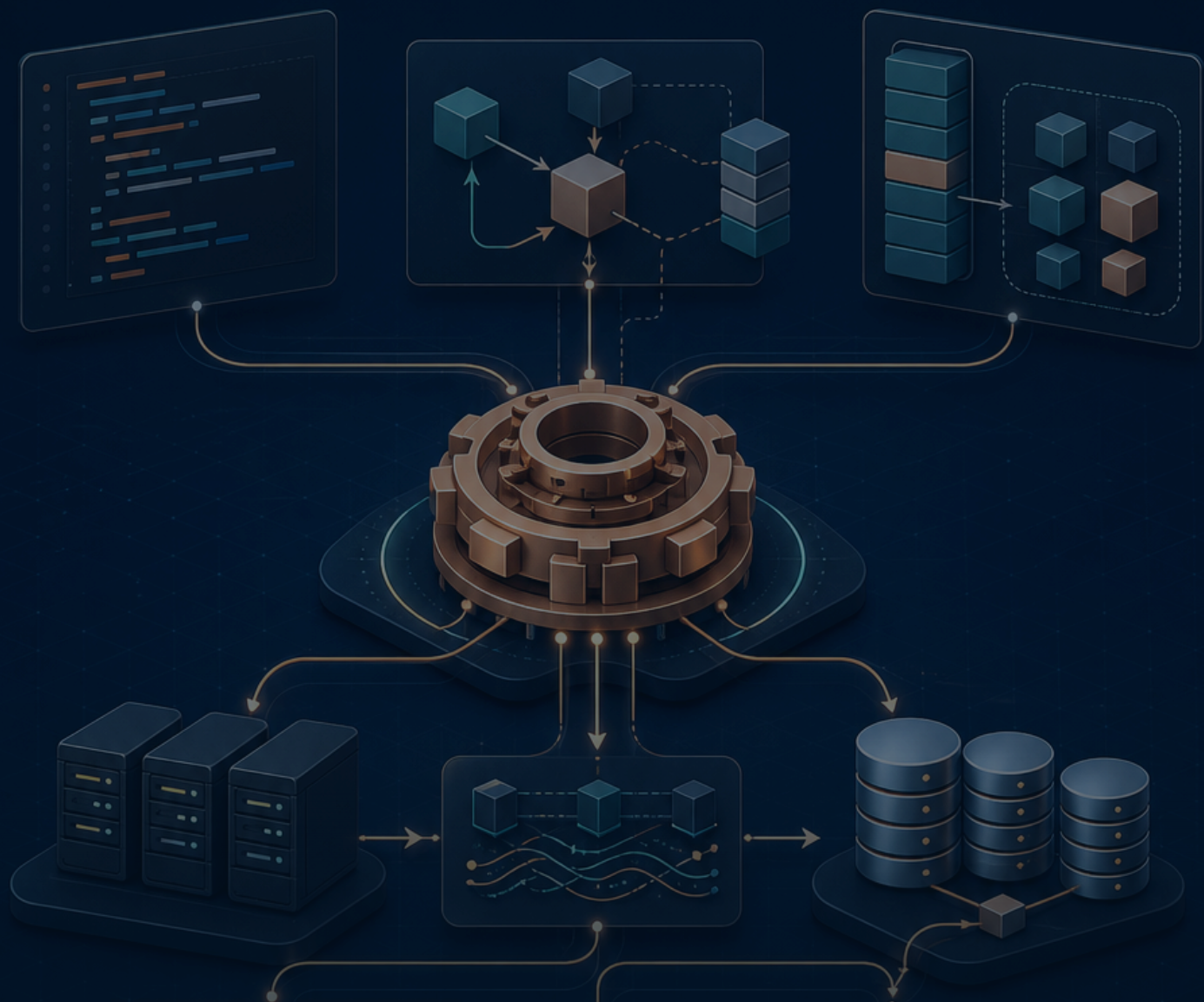




VOLUME 02 / RUST BACKEND MASTERY

Rust基礎 — 値・型・制御・データ構造

構文暗記ではなく、値がどの型で生まれ、変換され、分岐し、コレクションへ入るかを実行結果と一緒に学ぶ

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

小さなCLI例から始め、バックエンドの入力検証、状態表現、DB行変換につながる形でRustの基礎構文を身に付けます。

到達目標

- ✓ 値の型と変更可能性を説明できる
- ✓ Stringと&str、Vecとsliceを使い分けられる
- ✓ OptionとResultで欠損・失敗を表せる
- ✓ struct・enum・Iteratorで小さなドメイン処理を書ける

学習地図

01 変数・mut・シャドーイング・定数

Rustの変数は既定で不変です。値を変える意図はmutで宣言し、同じ名前を別の型へ変換するときはシャドーイングを使います。定数は型注釈とコンパイル時に決まる値が必要です。

02 整数・浮動小数・bool・charと変換

整数型は符号と幅、浮動小数は丸め誤差、charはUnicodeスカラー値を表します。asによる変換は切り捨てを起こせるため、外部入力ではTryFromやparseで失敗を扱います。

03 関数・文・式・戻り値

Rustではブロックもifもmatchも値を返す式です。末尾式にはセミコロンを付けず、早期returnは異常条件を先に除くために使います。関数の型署名を最初に読む習慣を付けます。

04 if・loop・while・for・match

制御構文は処理順序を決めます。ifの各枝は同じ型を返し、loopはbreakで値を返せます。matchはenumの全候補を網羅し、書き忘れをコンパイラに検出させます。

05 String・str・UTF-8

Stringは所有する可変UTF-8文字列、strは文字列スライスです。日本語は1文字が1バイトとは限らないため、bytes、chars、graphemeの違いを理解し、添字アクセスをしません。

06 タプル・配列・スライス・Vec

タプルは異なる型をひとまとめにし、配列は同じ型を固定長で、Vecは同じ型を可変長で持ちます。関数の読み取り入力は&[T]にすると配列とVecの両方を借りられます。

07 struct・impl・関連関数・メソッド

structは関連する値へ名前を付けてドメイン概念を表し、implはその型が守る操作を近くに置きます。生成時に不変条件を検証すれば、不正状態を後段へ流しにくくなります。

08 enum・Option・パターン分解

enumは取り得る状態を有限個に列挙し、各候補へ異なるデータを持たせられます。Option<T>は値がない可能性を型にし、null参照の代わりにSome/Noneをmatchします。

09 Result・エラー伝播・?演算子

Result<T,E>は成功値Tか失敗Eのどちらかです。?はErrを呼び出し元へ早期返却し、From変換があればエラー型もそろえます。unwrapは回復不能な不変条件に限定します。

10 HashMap・Iterator・closure・モジュール

HashMapはキーで値を引き、Iteratorは要素列を遅延変換し、closureは処理を値として渡します。モジュールは公開境界を作り、pubを必要最小限にします。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初の変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

letの不変性

STEP 2

mutの局所化

STEP 3

シャドーイング

STEP 4

constの契約

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

変数・mut・シャドーイング・定数

UNIT 01・CONCEPT

変数・mut・シャドーイング・定数

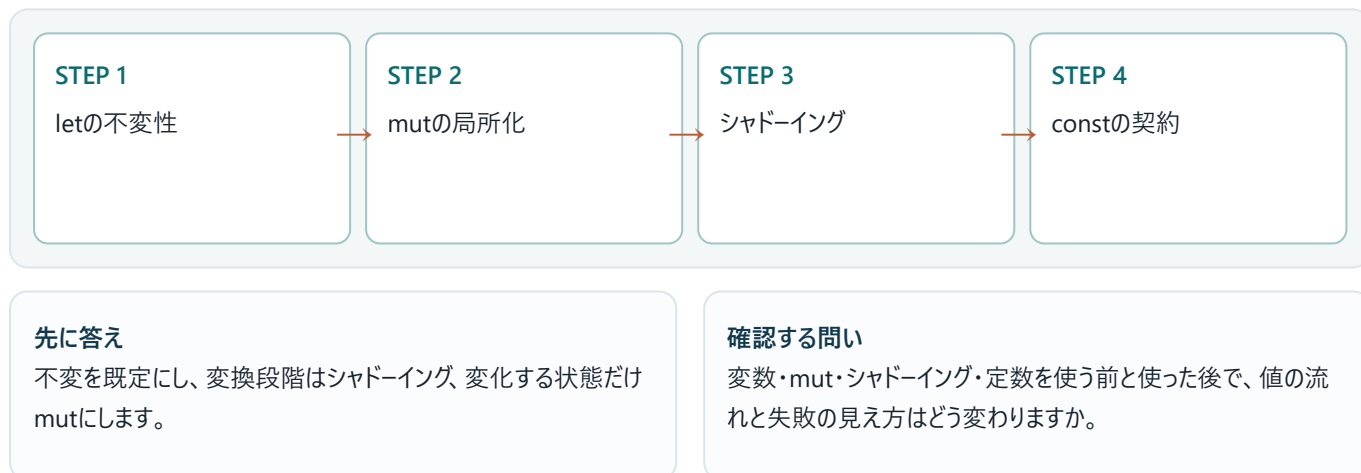
Rustの変数は既定で不変です。値を変える意図はmutで宣言し、同じ名前を別の型へ変換するときはシャドーイングを使います。定数は型注釈とコンパイル時に決まる値が必要です。

この単元の到達点

- letの不変性
- mutの局所化
- シャドーイング
- constの契約

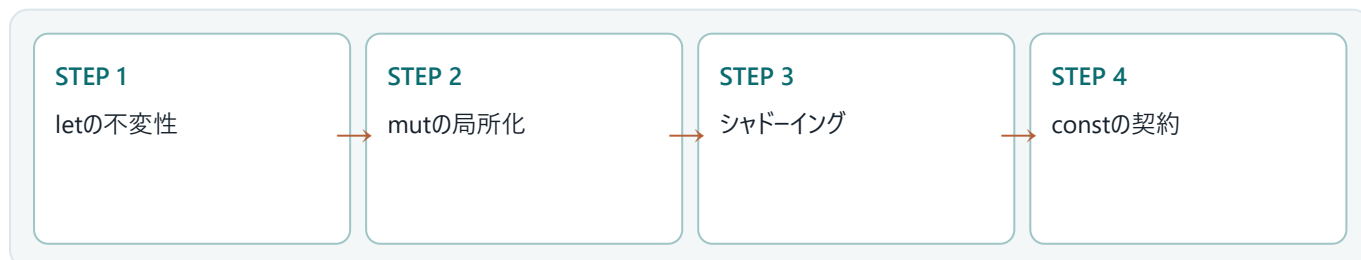
なぜバックエンドで必要か

設定値や入力を段階ごとに別の型へ変換し、可変状態は再試行回数など必要な狭い範囲へ閉じ込めます。



UNIT 01・MENTAL MODEL

変数・mut・シャドーイング・定数を図でつかむ



Rustの変数は既定で不変です。値を変える意図はmutで宣言し、同じ名前を別の型へ変換するときはシャドーイングを使います。定数は型注釈とコンパイル時に決まる値が必要です。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	letの不変性	letの不変性 — 入力と前提を確認し、境界を曖昧にしない。
2	mutの局所化	mutの局所化 — 値がどの順で変換されるかを追跡する。
3	シャドーイング	シャドーイング — 失敗を再現し、型またはログから原因を特定する。
4	constの契約	constの契約 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

変数・mut・シャドーイング・定数とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

変数・mut・シャドーイング・定数だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
const MAX_RETRIES: u8 = 3;
fn main() {
    let raw = " 42 ";
    let raw = raw.trim();
    let count: u32 = raw.parse().expect("number");
    let mut attempts = 0;
    attempts += 1;
    println!("count={count} attempts={attempts}/{MAX_RETRIES}");
}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>const MAX_RETRIES: u8 = 3;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let raw = " 42 ";</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let raw = raw.trim();</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let count: u32 = raw.parse().expect("number");</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力letの不変性の条件を満たすか確認する。
- ✓ mutの局所化を実行し、途中の型と状態を追う。
- ✓ シャドーイングで失敗を成功経路から分離する。
- ✓ constの契約により、出力と副作用を検証する。

型を声に出す 変数・mut・シャドーイング・定数に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

count=42 attempts=1/3

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn parse_port(raw: Option<String>) -> Result<u16, String> {
    let raw = raw.unwrap_or_else(|| "3000".into());
    raw.parse::<u16>().map_err(|_| format!("invalid PORT: {raw}"))
}
fn main() { assert_eq!(parse_port(None).unwrap(), 3000); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01・BACKEND APPLICATION

バックエンドのどこで使うか

設定値や入力を段階ごとに別の型へ変換し、可変状態は再試行回数など必要な狭い範囲へ閉じ込めます。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	letの不変性に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	mutの局所化の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	シャドーイングまたはconstの契約の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

変数・mut・シャドーイング・定数を再現して切り分ける

変数・mut・シャドーイング・定数の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: letの不変性
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: mutの局所化
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: シャドーイング
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: constの契約

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	letの不変性	letの不変性 — 入力と前提を確認し、境界を曖昧にしない。
2	mutの局所化	mutの局所化 — 値がどの順で変換されるかを追跡する。
3	シャドーイング	シャドーイング — 失敗を再現し、型またはログから原因を特定する。
4	constの契約	constの契約 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 不変を既定にし、変換段階はシャドーイング 変化する状態だけmutにします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01・PRACTICE

手を動かす演習

環境変数PORTを文字列からu16へ変換し、未設定・不正値・正常値を区別してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn parse_port(raw: Option<String>) -> Result<u16, String> {
    let raw = raw.unwrap_or_else(|| "3000".into());
    raw.parse::<u16>().map_err(|_| format!("invalid PORT: {raw}"))
}
fn main() { assert_eq!(parse_port(None).unwrap(), 3000); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 不変を既定にし、変換段階はシャドーイング 変化する状態だけmutにします。
- ✓ 変数・mut・シャドーイング・定数とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

整数・浮動小数・bool・charと変換

UNIT 02 · CONCEPT

整数・浮動小数・bool・charと変換

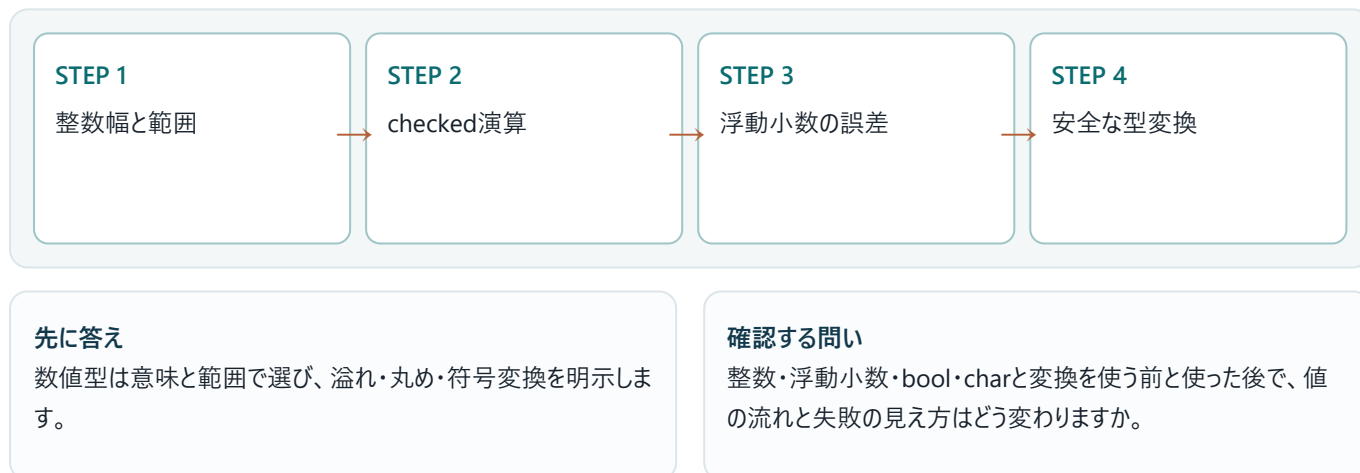
整数型は符号と幅、浮動小数は丸め誤差、charはUnicodeスカラー値を表します。asによる変換は切り捨てを起こせるため、外部入力ではTryFromやparseで失敗を扱います。

この単元の到達点

- 整数幅と範囲
- checked演算
- 浮動小数の誤差
- 安全な型変換

なぜバックエンドで必要か

金額はf64ではなく最小通貨単位の整数で持ち、IDや件数の変換は範囲外をResultで拒否します。



UNIT 02 · MENTAL MODEL

整数・浮動小数・bool・charと変換を図でつかむ



整数型は符号と幅、浮動小数は丸め誤差、charはUnicodeスカラー値を表します。asによる変換は切り捨てを起こせるため、外部入力ではTryFromやparseで失敗を扱います。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	整数幅と範囲	整数幅と範囲 — 入力と前提を確認し、境界を曖昧にしない。
2	checked演算	checked演算 — 値がどの順で変換されるかを追跡する。
3	浮動小数の誤差	浮動小数の誤差 — 失敗を再現し、型またはログから原因を特定する。
4	安全な型変換	安全な型変換 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

整数・浮動小数・bool・charと変換とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02 · MINIMUM CODE

まず動く最小コード

整数・浮動小数・bool・charと変換だけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
fn main() {
    let price_cents: u32 = 1_250;
    let quantity: u16 = 3;
    let total = price_cents.checked_mul(u32::from(quantity));
    println!("total={total:?}");
    println!("0.1 + 0.2 = {:.17}", 0.1_f64 + 0.2);
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let price_cents: u32 = 1_250;</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let quantity: u16 = 3;</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let total = price_cents.checked_mul(u32::from(quantity));</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>println!("{}", total);</code>	失敗時は早期returnし、呼び出し元へエラーを伝播します。

実行時に起きる順序

- ✓ 入力 that 整数幅と範囲の条件を満たすか確認する。
- ✓ checked演算を実行し、途中の型と状態を追う。
- ✓ 浮動小数の誤差で失敗を成功経路から分離する。
- ✓ 安全な型変換により、出力と副作用を検証する。

型を声に出す 整数・浮動小数・bool・charと変換に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
total=Some(3750)
0.1 + 0.2 = 0.30000000000000004
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力 that 処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn positive_id(value: i64) -> Result<u64, &'static str> {
    u64::try_from(value).map_err(|_| "id must be non-negative")
}
fn main() {
    assert_eq!(positive_id(7), Ok(7));
    assert!(positive_id(-1).is_err());
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

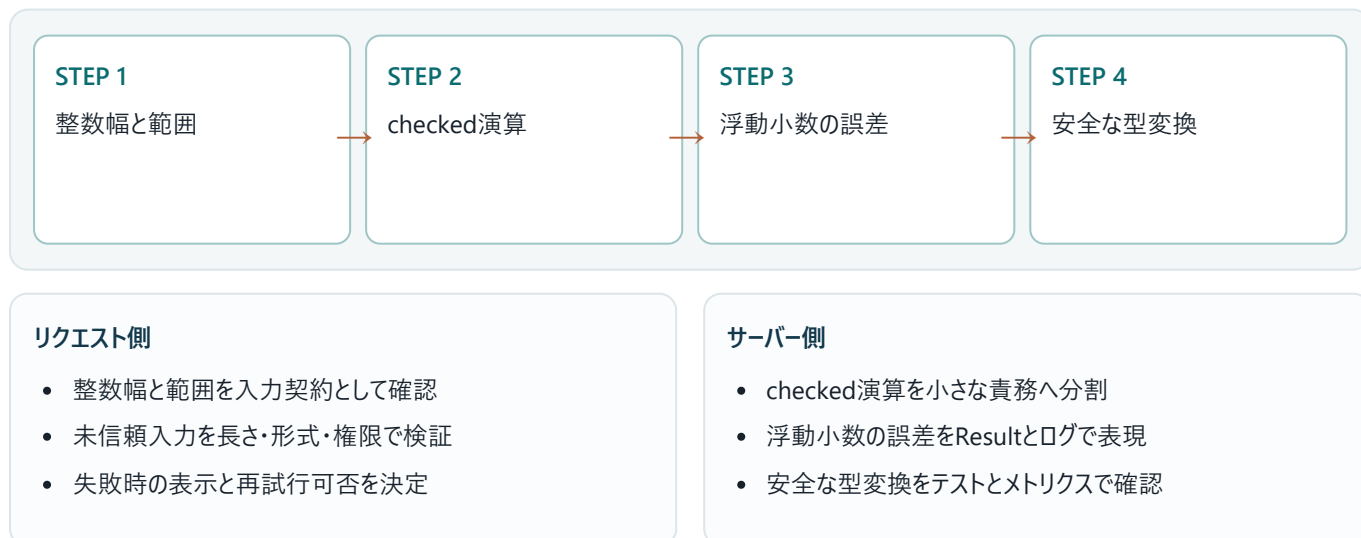
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 · BACKEND APPLICATION

バックエンドのどこで使うか

金額はf64ではなく最小通貨単位の整数で持ち、IDや件数の変換は範囲外をResultで拒否します。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	整数幅と範囲に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	checked演算の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	浮動小数の誤差または安全な型変換の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

整数・浮動小数・bool・charと変換を再現して切り分ける

整数・浮動小数・bool・charと変換の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 整数幅と範囲
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: checked演算
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 浮動小数の誤差
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 安全な型変換

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	整数幅と範囲	整数幅と範囲 — 入力と前提を確認し、境界を曖昧にしない。
2	checked演算	checked演算 — 値がどの順で変換されるかを追跡する。
3	浮動小数の誤差	浮動小数の誤差 — 失敗を再現し、型またはログから原因を特定する。
4	安全な型変換	安全な型変換 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 数値型は意味と範囲で選び、溢れ・丸め・符号変換を明示します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

i64のDB IDをu64へ安全に変換し、負数ならエラーにしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn positive_id(value: i64) -> Result<u64, &'static str> {
    u64::try_from(value).map_err(|_| "id must be non-negative")
}
fn main() {
    assert_eq!(positive_id(7), Ok(7));
    assert!(positive_id(-1).is_err());
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 数値型は意味と範囲で選び、溢れ・丸め・符号変換を明示します。
- ✓ 整数・浮動小数・bool・charと変換とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

関数・文・式・戻り値

UNIT 03 · CONCEPT

関数・文・式・戻り値

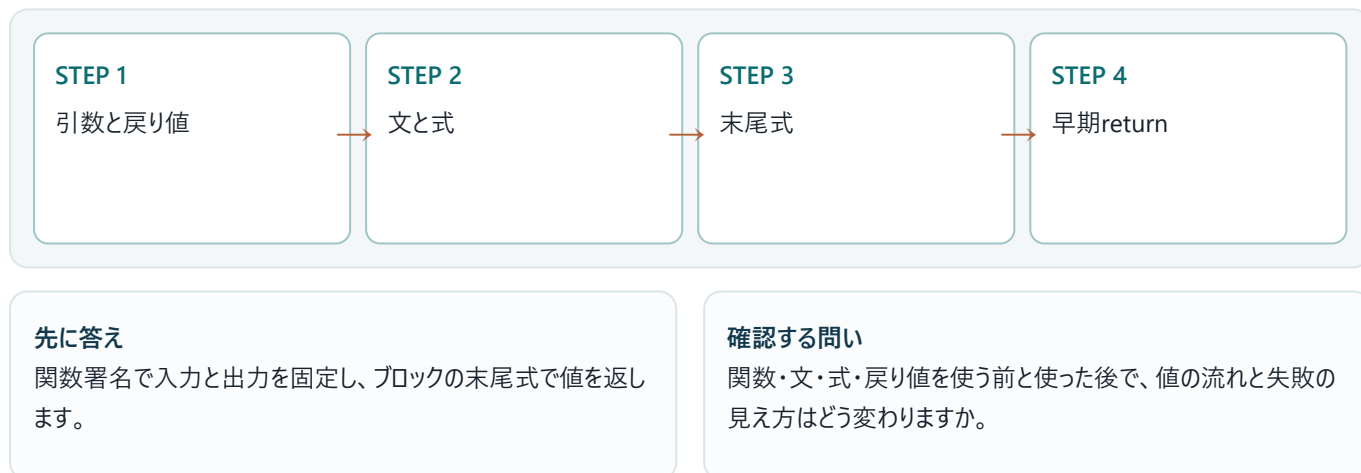
Rustではブロックもifもmatchも値を返す式です。末尾式にはセミコロンを付けず、早期returnは異常条件を先に除くために使います。関数の型署名を最初に読む習慣を付けます。

この単元の到達点

- 引数と戻り値
- 文と式
- 末尾式
- 早期return

なぜバックエンドで必要か

ハンドラを入力抽出、検証、ユースケース、レスポンス変換へ分けると、それぞれの関数署名が責務の境界になります。



UNIT 03 · MENTAL MODEL

関数・文・式・戻り値を図でつかむ



Rustではブロックもifもmatchも値を返す式です。末尾式にはセミコロンを付けず、早期returnは異常条件を先に除くために使います。関数の型署名を最初に読む習慣を付けます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	引数と戻り値	引数と戻り値 — 入力と前提を確認し、境界を曖昧にしない。
2	文と式	文と式 — 値がどの順で変換されるかを追跡する。
3	末尾式	末尾式 — 失敗を再現し、型またはログから原因を特定する。
4	早期return	早期return — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

関数・文・式・戻り値とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03 · MINIMUM CODE

まず動く最小コード

関数・文・式・戻り値だけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
fn normalize_title(raw: &str) -> String {
    let trimmed = raw.trim();
    if trimmed.is_empty() {
        return "Untitled".to_string();
    }
    trimmed.to_lowercase()
}
fn main() { println!("{}", normalize_title(" Rust ")); }
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn normalize_title(raw: &str) -> String {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let trimmed = raw.trim();</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>if trimmed.is_empty() {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>return "Untitled".to_string();</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が入数と戻り値の条件を満たすか確認する。
- ✓ 文と式を実行し、途中の型と状態を追う。
- ✓ 末尾式で失敗を成功経路から分離する。
- ✓ 早期returnにより、出力と副作用を検証する。

型を声に出す 関数・文・式・戻り値に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

rust

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn validate_title(raw: &str) -> Result<&str, &'static str> {
    let title = raw.trim();
    if title.is_empty() { return Err("title is required"); }
    if title.chars().count() > 80 { return Err("title is too long"); }
    Ok(title)
}

fn main() { assert_eq!(validate_title(" Rust "), Ok("Rust")); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

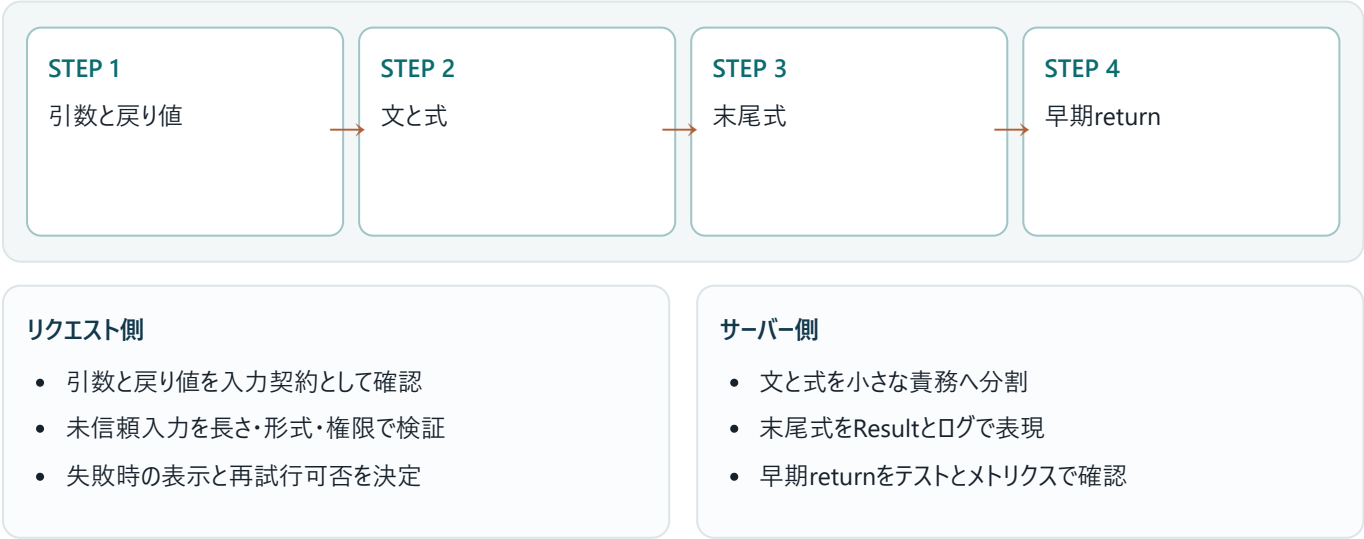
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03 · BACKEND APPLICATION

バックエンドのどこで使うか

ハンドラを入力抽出、検証、ユースケース、レスポンス変換へ分けると、それぞれの関数署名が責務の境界になります。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	引数と戻り値に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	文と式の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	末尾式または早期returnの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

関数・文・式・戻り値を再現して切り分ける

関数・文・式・戻り値の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 引数と戻り値
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 文と式
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 末尾式
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 早期return

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	引数と戻り値	引数と戻り値 — 入力と前提を確認し、境界を曖昧にしない。
2	文と式	文と式 — 値がどの順で変換されるかを追跡する。
3	末尾式	末尾式 — 失敗を再現し、型またはログから原因を特定する。
4	早期return	早期return — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 関数署名で入力と出力を固定し、ブロックの末尾式で値を返します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

空白だけ、通常文字列、長すぎる文字列を扱う`validate_title`を`Result`で作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時に`panic`せず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn validate_title(raw: &str) -> Result<&str, &'static str> {  
    let title = raw.trim();  
    if title.is_empty() { return Err("title is required"); }  
    if title.chars().count() > 80 { return Err("title is too long"); }  
    Ok(title)  
}  
  
fn main() { assert_eq!(validate_title(" Rust "), Ok("Rust")); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 関数署名で入力と出力を固定し、ブロックの末尾式で値を返します。
- ✓ 関数・文・式・戻り値とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

if・loop・while・for・match

UNIT 04 ・ CONCEPT

if・loop・while・for・match

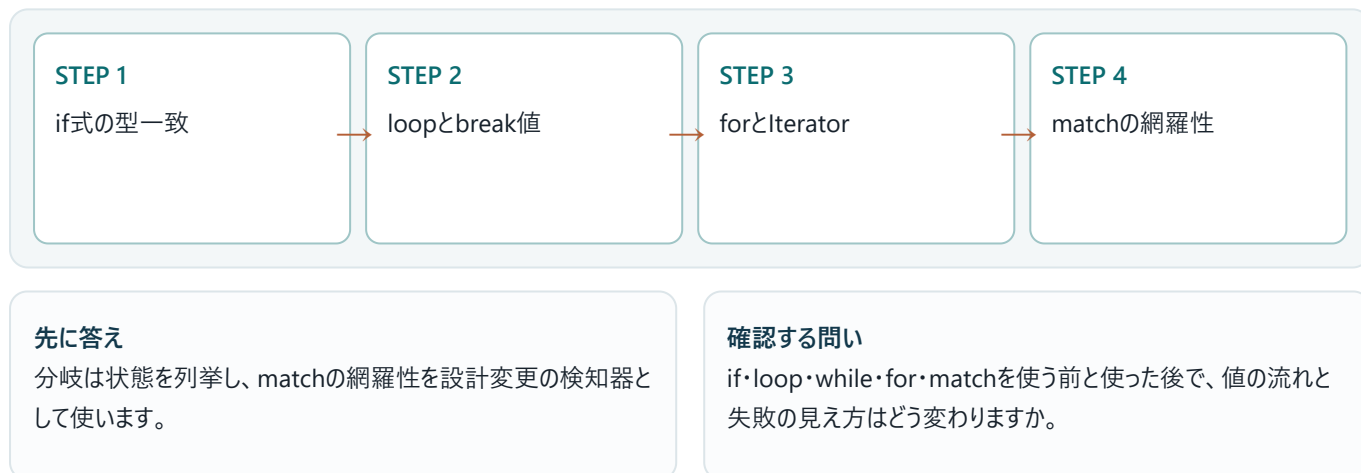
制御構文は処理順序を決めます。ifの各枝は同じ型を返し、loopはbreakで値を返せます。matchはenumの全候補を網羅し、書き忘れをコンパイラに検出させます。

この単元の到達点

- if式の型一致
- loopとbreak値
- forとIterator
- matchの網羅性

なぜバックエンドで必要か

HTTP状態、権限、DB結果をmatchで分類し、想定外を明示します。無条件の_で重要な新規状態を隠さない判断も必要です。



UNIT 04 ・ MENTAL MODEL

if・loop・while・for・matchを図でつかむ



制御構文は処理順序を決めます。ifの各枝は同じ型を返し、loopはbreakで値を返せます。matchはenumの全候補を網羅し、書き忘れをコンパイラに検出させます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	if式の型一致	if式の型一致 — 入力と前提を確認し、境界を曖昧にしない。
2	loopとbreak値	loopとbreak値 — 値がどの順で変換されるかを追跡する。
3	forとIterator	forとIterator — 失敗を再現し、型またはログから原因を特定する。
4	matchの網羅性	matchの網羅性 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

if・loop・while・for・matchとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE

まず動く最小コード

if・loop・while・for・matchだけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
fn status_label(code: u16) -> &'static str {
    match code {
        200..=299 => "success",
        400..=499 => "client error",
        500..=599 => "server error",
        _ => "other",
    }
}

fn main() { for code in [200, 404, 503] { println!("{code}: {}", status_label(code)); } }
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn status_label(code: u16) -> &'static str {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>match code {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>200..=299 => "success",</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>400..=499 => "client error",</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>500..=599 => "server error",</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がif式の型一致の条件を満たすか確認する。
- ✓ loopとbreak値を実行し、途中の型と状態を追う。
- ✓ forとiteratorで失敗を成功経路から分離する。
- ✓ matchの網羅性により、出力と副作用を検証する。

型を声に出す if・loop・while・for・matchに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

200: success
404: client error
503: server error

見える結果
端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Debug, Clone, Copy)]
enum NoteState { Draft, Published, Archived }
fn publish(state: NoteState) -> Result<NoteState, &'static str> {
    match state {
        NoteState::Draft => Ok(NoteState::Published),
        _ => Err("only draft can be published"),
    }
}
fn main() { assert!(publish(NoteState::Draft).is_ok()); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

HTTP状態、権限、DB結果をmatchで分類し、想定外を明示します。無条件の_で重要な新規状態を隠さない判断も必要です。



リクエスト側

- if式の型一致を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- loopとbreak値を小さな責務へ分割
- forとIteratorをResultとログで表現
- matchの網羅性をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	if式の型一致に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	loopとbreak値の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	forとIteratorまたはmatchの網羅性の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

if・loop・while・for・matchを再現して切り分ける

if・loop・while・for・matchの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: if式の型一致
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: loopとbreak値
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: forとIterator
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: matchの網羅性

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	if式の型一致	if式の型一致 — 入力と前提を確認し、境界を曖昧にしない。
2	loopとbreak値	loopとbreak値 — 値がどの順で変換されるかを追跡する。
3	forとIterator	forとIterator — 失敗を再現し、型またはログから原因を特定する。
4	matchの網羅性	matchの網羅性 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 分岐は状態を列挙し、matchの網羅性を設計変更の検知器として使います。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 · PRACTICE

手を動かす演習

NoteStateをDraft、Published、Archivedで定義し、許可される状態遷移だけをResultで返してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Debug, Clone, Copy)]
enum NoteState { Draft, Published, Archived }
fn publish(state: NoteState) -> Result<NoteState, &'static str> {
    match state {
        NoteState::Draft => Ok(NoteState::Published),
        _ => Err("only draft can be published"),
    }
}
fn main() { assert!(publish(NoteState::Draft).is_ok()); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 分岐は状態を列挙し、matchの網羅性を設計変更の検知器として使います。
- ✓ if・loop・while・for・matchとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

String・str・UTF-8

UNIT 05 ・ CONCEPT

String・str・UTF-8

Stringは所有する可変UTF-8文字列、strは文字列スライスです。日本語は1文字が1バイトとは限らないため、bytes、chars、graphemeの違いを理解し、添字アクセスをしません。

この単元の到達点

- Stringの所有
- &strの借用
- UTF-8とbytes
- 文字数と切り出し

なぜバックエンドで必要か

フォーム入力、URL、DB TEXTはUTF-8文字列です。長さ制限をbytesか文字数のどちらで課するか、DBや外部APIの契約に合わせます。

入力・所有する側

Stringの所有

owner

&strの借用

owner

処理・借りる側

UTF-8とbytes

borrow

文字数と切り出し

borrow

先に答え

所有が不要なら&strで借り、保存や返却に必要な場所でStringへ変換します。

確認する問い

String・str・UTF-8を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 05 ・ MENTAL MODEL

String・str・UTF-8を図でつかむ

入力・所有する側

Stringの所有

owner

&strの借用

owner

処理・借りる側

UTF-8とbytes

borrow

文字数と切り出し

borrow

Stringは所有する可変UTF-8文字列、strは文字列スライスです。日本語は1文字が1バイトとは限らないため、bytes、chars、graphemeの違いを理解し、添字アクセスをしません。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Stringの所有	Stringの所有 — 入力と前提を確認し、境界を曖昧にしない。
2	&strの借用	&strの借用 — 値がどの順で変換されるかを追跡する。
3	UTF-8とbytes	UTF-8とbytes — 失敗を再現し、型またはログから原因を特定する。
4	文字数と切り出し	文字数と切り出し — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

String・str・UTF-8とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05 · MINIMUM CODE まず動く最小コード

String・str・UTF-8だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn summarize(title: &str) -> String {  
    let preview: String = title.chars().take(5).collect();  
    format!("{preview}... (bytes={}, chars={})", title.len(), title.chars().count())  
}  
fn main() { println!("{}", summarize("Rust入門ノート")); }
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn summarize(title: &str) -> String {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let preview: String = title.chars().take(5).collect();</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>format!("{preview}... (bytes={}, chars={})", title.len(), title.chars().count</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main() { println!("{}", summarize("Rust入門ノ ット")); }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力Stringの所有の条件を満たすか確認する。
- ✓ &strの借用を実行し、途中の型と状態を追う。
- ✓ UTF-8とbytesで失敗を成功経路から分離する。
- ✓ 文字数と切り出しにより、出力と副作用を検証する。

型を声に出す String・str・UTF-8に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

Rust入... (bytes=19, chars=9)

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn normalize_title(raw: &str) -> Result<String, &'static str> {
    let title = raw.trim();
    let len = title.chars().count();
    if !(1..=80).contains(&len) { return Err("title length must be 1..=80"); }
    Ok(title.to_owned())
}

fn main() { assert_eq!(normalize_title(" Rust "), Ok("Rust".into())); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

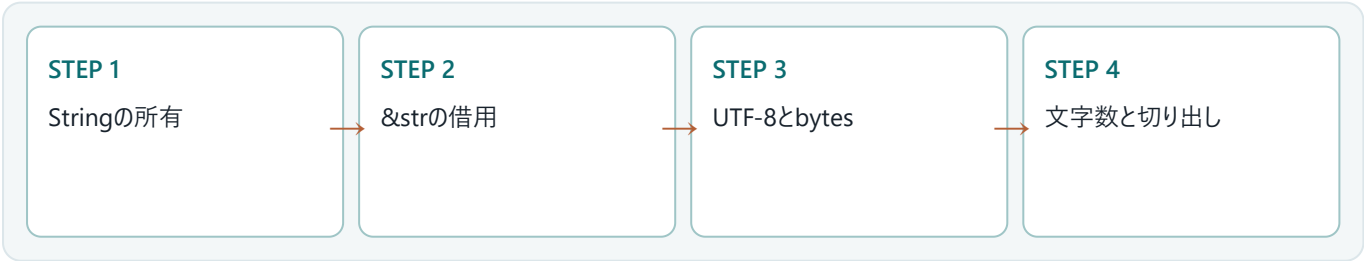
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05・BACKEND APPLICATION

バックエンドのどこで使うか

フォーム入力、URL、DB TEXTはUTF-8文字列です。長さ制限をbytesか文字数のどちらで課すか、DBや外部APIの契約に合わせます。



リクエスト側

- Stringの所有を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- &strの借用を小さな責務へ分割
- UTF-8とbytesをResultとログで表現
- 文字数と切り出しをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Stringの所有に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	&strの借用の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	UTF-8とbytesまたは文字数と切り出しの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

String・str・UTF-8を再現して切り分ける

String・str・UTF-8の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Stringの所有
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: &strの借用
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: UTF-8とbytes
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 文字数と切り出し

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Stringの所有	Stringの所有 — 入力と前提を確認し、境界を曖昧にしない。
2	&strの借用	&strの借用 — 値がどの順で変換されるかを追跡する。
3	UTF-8とbytes	UTF-8とbytes — 失敗を再現し、型またはログから原因を特定する。
4	文字数と切り出し	文字数と切り出し — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 所有が不要なら&strで借り、保存や返却に必要な場所でStringへ変換します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

タイトルを前後空白除去し、1〜80文字なら所有するStringを返す関数を作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn normalize_title(raw: &str) -> Result<String, &'static str> {
    let title = raw.trim();
    let len = title.chars().count();
    if !(1..=80).contains(&len) { return Err("title length must be 1..=80"); }
    Ok(title.to_owned())
}

fn main() { assert_eq!(normalize_title(" Rust "), Ok("Rust".into())); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 所有が不要なら&strで借り、保存や返却に必要な場所でStringへ変換します。
- ✓ String・str・UTF-8とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

タプル・配列・スライス・Vec

UNIT 06 · CONCEPT

タプル・配列・スライス・Vec

タプルは異なる型をひとまとめにし、配列は同じ型を固定長で、Vecは同じ型を可変長で持ちます。関数の読み取り入力
は`&[T]`にすると配列とVecの両方を借りられます。

この単元の到達点

- タプルと分解
- 固定長配列
- スライス`&[T]`
- 可変長Vec`<T>`

なぜバックエンドで必要か

DBから取得した複数行はVec`<Model>`、関数へ読むだけで渡すときは`&[Model]`にし、所有権移動と不要な割り当てを避けます。

入力・所有する側

タプルと分解

owner

固定長配列

owner

処理・借りる側

スライス`&[T]`

borrow

可変長Vec`<T>`

borrow

先に答え

保存形はVec、読み取り専用の関数境界は`&[T]`を第一候補に
します。

確認する問い

タプル・配列・スライス・Vecを使う前と使った後で、値の流れと失
敗の見え方はどう変わりますか。

UNIT 06 · MENTAL MODEL

タプル・配列・スライス・Vecを図でつかむ

入力・所有する側

タプルと分解

owner

固定長配列

owner

処理・借りる側

スライス`&[T]`

borrow

可変長Vec`<T>`

borrow

タプルは異なる型をひとまとめにし、配列は同じ型を固定長で、Vecは同じ型を可変長で持ちます。関数の読み取り入力は`&[T]`にすると配列
とVecの両方を借りられます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認
します。

順序	観察対象	初心者が確認すること
1	タプルと分解	タプルと分解 — 入力と前提を確認し、境界を曖昧にしない。
2	固定長配列	固定長配列 — 値がどの順で変換されるかを追跡する。
3	スライス&[T]	スライス&[T] — 失敗を再現し、型またはログから原因を特定する。
4	可変長Vec<T>	可変長Vec<T> — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

タプル・配列・スライス・Vecとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06 · MINIMUM CODE

まず動く最小コード

タプル・配列・スライス・Vecだけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
fn average(values: &[i32]) -> Option<f64> {
    if values.is_empty() { return None; }
    let sum: i32 = values.iter().sum();
    Some(f64::from(sum) / values.len() as f64)
}
fn main() {
    let scores = vec![80, 90, 100];
    println!("{:?}", average(&scores));
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn average(values: &[i32]) -> Option<f64> {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>if values.is_empty() { return None; }</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>let sum: i32 = values.iter().sum();</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>Some(f64::from(sum) / values.len() as f64)</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がタプルと分解の条件を満たすか確認する。
- ✓ 固定長配列を実行し、途中の型と状態を追う。
- ✓ スライス&[T]で失敗を成功経路から分離する。
- ✓ 可変長Vec<T>により、出力と副作用を検証する。

型を声に出す タプル・配列・スライス・Vecに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

Some(90.0)

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn total(ids: &[u64]) -> u64 { ids.iter().sum() }
fn main() {
    let fixed = [1, 2, 3];
    let dynamic = vec![4, 5];
    assert_eq!(total(&fixed), 6);
    assert_eq!(total(&dynamic), 9);
    assert_eq!(total(&[]), 0);
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

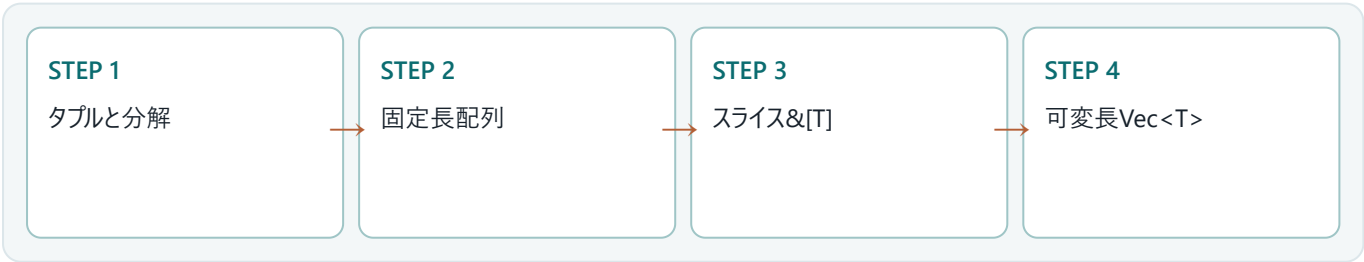
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06・BACKEND APPLICATION

バックエンドのどこで使うか

DBから取得した複数行はVec<Model>、関数へ読むだけで渡すときは&[Model]にし、所有権移動と不要な割り当てを避けます。



リクエスト側

- タプルと分解を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- 固定長配列を小さな責務へ分割
- スライス&[T]をResultとログで表現
- 可変長Vec<T>をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	タプルと分解に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違ふ	固定長配列の入力または実行順序が想定と違ふ	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	スライス&[T]または可変長Vec<T>の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

タプル・配列・スライス・Vecを再現して切り分ける

タプル・配列・スライス・Vecの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: タプルと分解
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 固定長配列
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: スライス&[T]
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 可変長Vec<T>

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	タプルと分解	タプルと分解 — 入力と前提を確認し、境界を曖昧にしない。
2	固定長配列	固定長配列 — 値がどの順で変換されるかを追跡する。
3	スライス&[T]	スライス&[T] — 失敗を再現し、型またはログから原因を特定する。
4	可変長Vec<T>	可変長Vec<T> — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 保存形はVec、読み取り専用の関数境界は&[T]を第一候補にします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

&[u64]を受け取り重複を除かず合計する関数と、空配列を含むテストを作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn total(ids: &[u64]) -> u64 { ids.iter().sum() }
fn main() {
    let fixed = [1, 2, 3];
    let dynamic = vec![4, 5];
    assert_eq!(total(&fixed), 6);
    assert_eq!(total(&dynamic), 9);
    assert_eq!(total(&[]), 0);
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 保存形はVec、読み取り専用の関数境界は&[T]を第一候補にします。
- ✓ タプル・配列・スライス・Vecとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

struct・impl・関連関数・メソッド

UNIT 07・CONCEPT

struct・impl・関連関数・メソッド

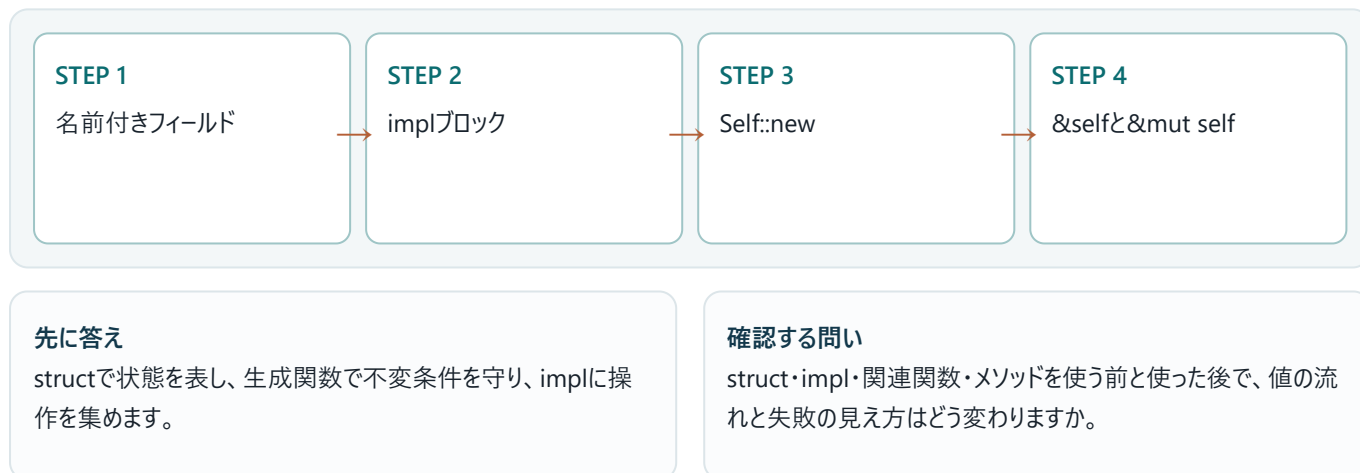
structは関連する値へ名前を付けてドメイン概念を表し、implはその型が守る操作を近くに置きます。生成時に不変条件を検証すれば、不正状態を後段へ流しにくくなります。

この単元の到達点

- 名前付きフィールド
- implブロック
- Self::new
- &selfと&mut self

なぜバックエンドで必要か

User、Note、Sessionをstructで表し、SQL行、フォームDTO、ドメイン型を分けると、どの値が検証済みかを型で追えます。



UNIT 07・MENTAL MODEL

struct・impl・関連関数・メソッドを図でつかむ



structは関連する値へ名前を付けてドメイン概念を表し、implはその型が守る操作を近くに置きます。生成時に不変条件を検証すれば、不正状態を後段へ流しにくくなります。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	名前付きフィールド	名前付きフィールド — 入力と前提を確認し、境界を曖昧にしない。
2	implブロック	implブロック — 値がどの順で変換されるかを追跡する。
3	Self::new	Self::new — 失敗を再現し、型またはログから原因を特定する。
4	&selfと&mut self	&selfと&mut self — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

struct・impl・関連関数・メソッドとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07 · MINIMUM CODE

まず動く最小コード

struct・impl・関連関数・メソッドだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[derive(Debug)]
struct Note { title: String, done: bool }
impl Note {
    fn new(title: impl Into<String>) -> Self {
        Self { title: title.into(), done: false }
    }
    fn complete(&mut self) { self.done = true; }
}
fn main() { let mut note = Note::new("ownership"); note.complete(); println!("{note:?}"); }
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Debug)]</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>struct Note { title: String, done: bool }</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>impl Note {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn new(title: impl Into<String>) -> Self {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>Self { title: title.into(), done: false }</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が名前付きフィールドの条件を満たすか確認する。
- ✓ `impl`ブロックを実行し、途中の型と状態を追う。
- ✓ `Self::new`で失敗を成功経路から分離する。
- ✓ `&self`と`&mut self`により、出力と副作用を検証する。

型を声に出す `struct`・`impl`・関連関数・メソッドに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

Note { title: "ownership", done: true }

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Debug)]
struct Note { title: String, done: bool }
impl Note {
    fn new(raw: &str) -> Result<Self, &'static str> {
        let title = raw.trim();
        if title.is_empty() { return Err("title required"); }
        Ok(Self { title: title.into(), done: false })
    }
    fn toggle(&mut self) { self.done = !self.done; }
}
fn main() { assert!(Note::new(" ").is_err()); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

User、Note、Sessionをstructで表し、SQL行、フォームDTO、ドメイン型を分けると、どの値が検証済みかを型で追えます。



リクエスト側

- 名前付きフィールドを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- implブロックを小さな責務へ分割
- Self::newをResultとログで表現
- &selfと&mut selfをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	名前付きフィールドに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	implブロックの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	Self::newまたは&selfと&mut selfの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

struct・impl・関連関数・メソッドを再現して切り分ける

struct・impl・関連関数・メソッドの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 名前付きフィールド
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: implブロック
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: Self::new
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: &selfと&mut self

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	名前付きフィールド	名前付きフィールド — 入力と前提を確認し、境界を曖昧にしない。
2	implブロック	implブロック — 値がどの順で変換されるかを追跡する。
3	Self::new	Self::new — 失敗を再現し、型またはログから原因を特定する。
4	&selfと&mut self	&selfと&mut self — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 structで状態を表し、生成関数で不変条件を守り、implに操作を集めます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

空タイトルを拒否する`Note::new`と、完了状態を戻せる`toggle`を実装してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Debug)]
struct Note { title: String, done: bool }
impl Note {
    fn new(raw: &str) -> Result<Self, &'static str> {
        let title = raw.trim();
        if title.is_empty() { return Err("title required"); }
        Ok(Self { title: title.into(), done: false })
    }
    fn toggle(&mut self) { self.done = !self.done; }
}
fn main() { assert!(Note::new(" ").is_err()); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ structで状態を表し、生成関数で不変条件を守り、implに操作を集めます。
- ✓ struct・impl・関連関数・メソッドとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

enum・Option・パターン分解

UNIT 08・CONCEPT

enum・Option・パターン分解

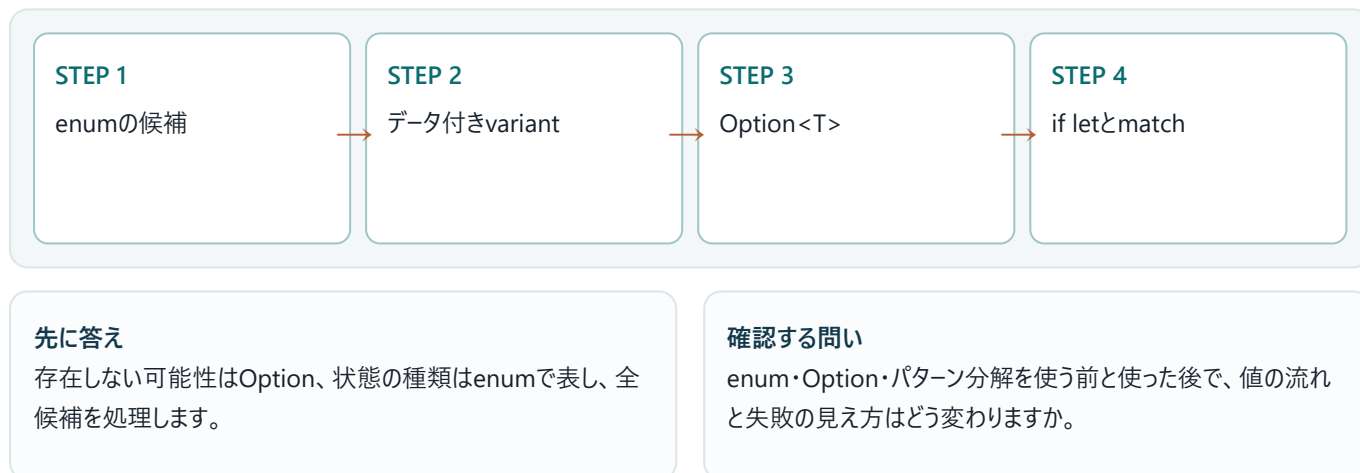
enumは取り得る状態を有限個に列挙し、各候補へ異なるデータを持たせられます。Option<T>は値がない可能性を型にし、null参照の代わりにSome/Noneをmatchします。

この単元の到達点

- enumの候補
- データ付きvariant
- Option<T>
- if letとmatch

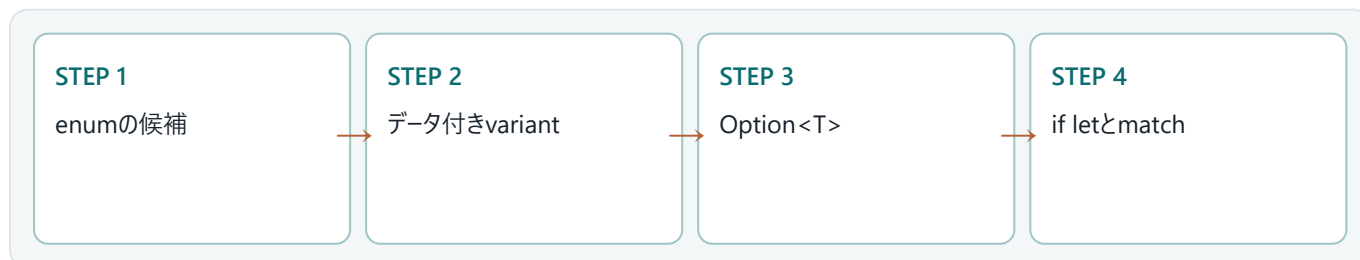
なぜバックエンドで必要か

DBの0件をOption、権限不足を別のエラーvariantとして区別すると、HTTP 404と403を正しく返せます。



UNIT 08・MENTAL MODEL

enum・Option・パターン分解を図でつかむ



enumは取り得る状態を有限個に列挙し、各候補へ異なるデータを持たせられます。Option<T>は値がない可能性を型にし、null参照の代わりにSome/Noneをmatchします。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	enumの候補	enumの候補 — 入力と前提を確認し、境界を曖昧にしない。
2	データ付きvariant	データ付きvariant — 値がどの順で変換されるかを追跡する。
3	Option<T>	Option<T> — 失敗を再現し、型またはログから原因を特定する。
4	if letとmatch	if letとmatch — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

enum・Option・パターン分解とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08 ・ MINIMUM CODE

まず動く最小コード

enum・Option・パターン分解だけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
#[derive(Debug)]
enum Lookup { Found { id: i64, title: String }, Missing }
fn find(id: i64) -> Lookup {
    if id == 1 { Lookup::Found { id, title: "Rust".into() } } else { Lookup::Missing }
}
fn main() {
    match find(1) {
        Lookup::Found { id, title } => println!("{id}: {title}"),
        Lookup::Missing => println!("not found"),
    }
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08 ・ LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Debug)]</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>enum Lookup { Found { id: i64, title: String }, Missing }</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn find(id: i64) -> Lookup {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>if id == 1 { Lookup::Found { id, title: "Rust".into() } } else { Lookup::M</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がenumの候補の条件を満たすか確認する。
- ✓ データ付きvariantを実行し、途中の型と状態を追う。
- ✓ Option<T>で失敗を成功経路から分離する。
- ✓ if letとmatchにより、出力と副作用を検証する。

型を声に出す enum・Option・パターン分解に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

1: Rust

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn normalize_query(raw: Option<&str>) -> Option<String> {
    raw.map(str::trim).filter(|s| !s.is_empty()).map(str::to_owned)
}
fn main() {
    assert_eq!(normalize_query(Some(" Rust ")), Some("Rust".into()));
    assert_eq!(normalize_query(Some("  ")), None);
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

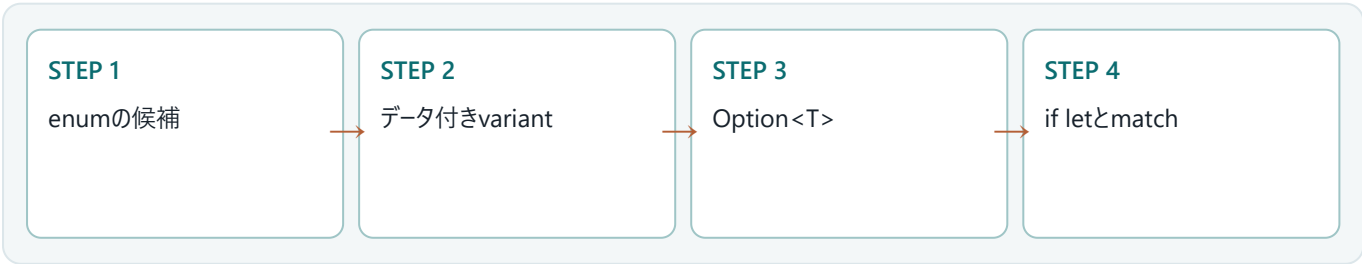
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08・BACKEND APPLICATION

バックエンドのどこで使うか

DBの0件をOption、権限不足を別のエラーvariantとして区別すると、HTTP 404と403を正しく返せます。



リクエスト側

- enumの候補を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- データ付きvariantを小さな責務へ分割
- Option<T>をResultとログで表現
- if letとmatchをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	enumの候補に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	データ付きvariantの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	Option<T>またはif letとmatchの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

enum・Option・パターン分解を再現して切り分ける

enum・Option・パターン分解の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: enumの候補
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: データ付きvariant
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: Option<T>
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: if letとmatch

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	enumの候補	enumの候補 — 入力と前提を確認し、境界を曖昧にしない。
2	データ付きvariant	データ付きvariant — 値がどの順で変換されるかを追跡する。
3	Option<T>	Option<T> — 失敗を再現し、型またはログから原因を特定する。
4	if letとmatch	if letとmatch — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 存在しない可能性はOption、状態の種類はenumで表し、全候補を処理します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

検索語がNone、空、通常文字列の三状態を正規化し、空はNoneとして扱ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn normalize_query(raw: Option<&str>) -> Option<String> {
    raw.map(str::trim).filter(|s| !s.is_empty()).map(str::to_owned)
}
fn main() {
    assert_eq!(normalize_query(Some(" Rust ")), Some("Rust".into()));
    assert_eq!(normalize_query(Some("  ")), None);
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 存在しない可能性はOption、状態の種類はenumで表し、全候補を処理します。
- ✓ enum・Option・パターン分解とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

Result・エラー伝播・?演算子

UNIT 09 · CONCEPT

Result・エラー伝播・?演算子

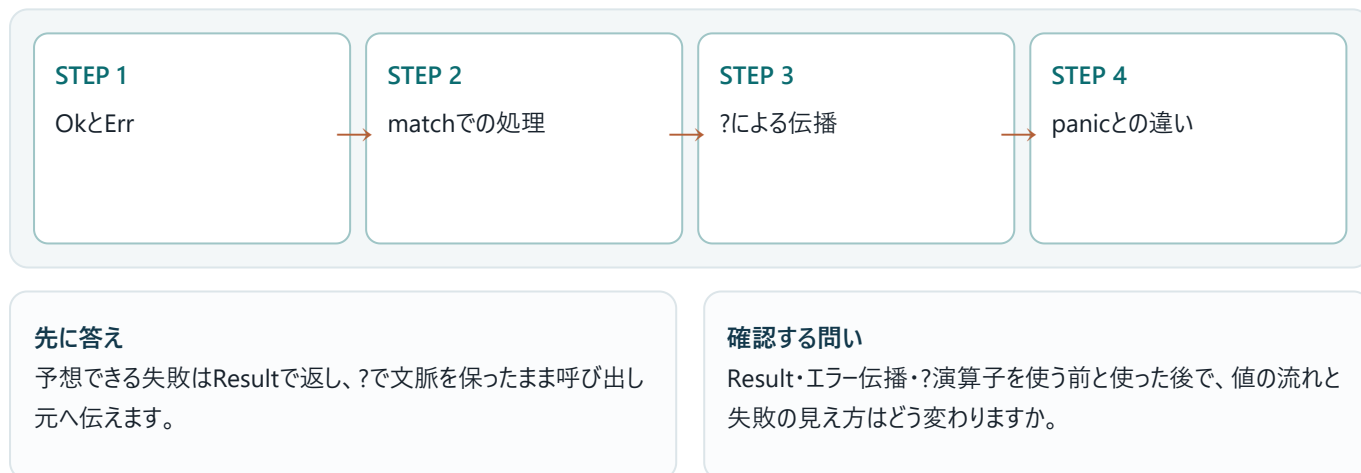
Result<T,E>は成功値Tか失敗Eのどちらかです。?はErrを呼び出し元へ早期返却し、From変換があればエラー型もそろえます。unwrapは回復不能な不変条件に限定します。

この単元の到達点

- OkとErr
- matchでの処理
- ?による伝播
- panicとの違い

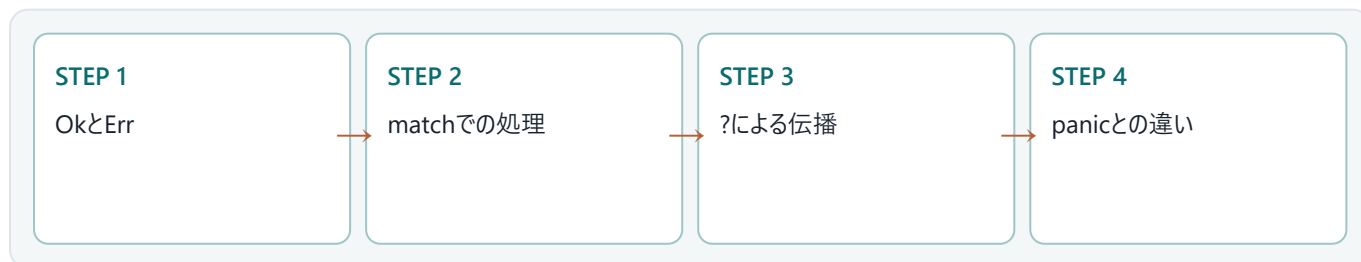
なぜバックエンドで必要か

HTTP入力、DB、外部APIはすべて失敗します。層ごとに文脈を加え、利用者向けメッセージと内部ログを分けます。



UNIT 09 · MENTAL MODEL

Result・エラー伝播・?演算子を図でつかむ



Result<T,E>は成功値Tか失敗Eのどちらかです。?はErrを呼び出し元へ早期返却し、From変換があればエラー型もそろえます。unwrapは回復不能な不変条件に限定します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	OkとErr	OkとErr — 入力と前提を確認し、境界を曖昧にしない。
2	matchでの処理	matchでの処理 — 値がどの順で変換されるかを追跡する。
3	?による伝播	?による伝播 — 失敗を再現し、型またはログから原因を特定する。
4	panicとの違い	panicとの違い — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Result・エラー伝播・?演算子とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09 · MINIMUM CODE

まず動く最小コード

Result・エラー伝播・?演算子だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use std::num::ParseIntError;
fn parse_limit(raw: &str) -> Result<u32, ParseIntError> {
    let value = raw.trim().parse::<u32>()?;
    Ok(value.min(100))
}
fn main() {
    println!("{:?}", parse_limit("250"));
    println!("{:?}", parse_limit("many"));
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use std::num::ParseIntError;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn parse_limit(raw: &str) -> Result<u32, ParseIntError> {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let value = raw.trim().parse::<u32>()?;</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>Ok(value.min(100))</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がOkとErrの条件を満たすか確認する。
- ✓ matchでの処理を実行し、途中の型と状態を追う。
- ✓ ?による伝播で失敗を成功経路から分離する。
- ✓ panicとの違いにより、出力と副作用を検証する。

型を声に出す Result・エラー伝播・?演算子に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
Ok(100)
Err(ParseIntError { kind: InvalidDigit })
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Debug)]
enum ConfigError { Missing, Invalid, OutOfRange }
fn port(raw: Option<&str>) -> Result<u16, ConfigError> {
    let text = raw.ok_or(ConfigError::Missing)?;
    let value = text.parse::<u32>().map_err(|_| ConfigError::Invalid)?;
    u16::try_from(value).map_err(|_| ConfigError::OutOfRange)
}
fn main() { assert!(matches!(port(Some("70000")), Err(ConfigError::OutOfRange))); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

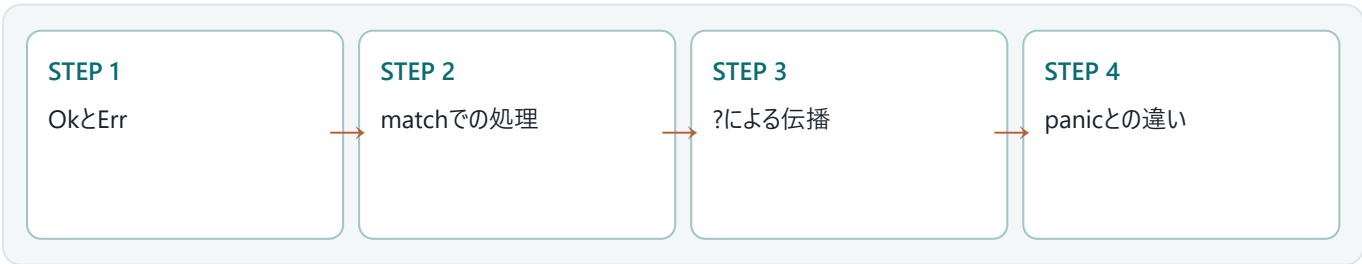
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09・BACKEND APPLICATION

バックエンドのどこで使うか

HTTP入力、DB、外部APIはすべて失敗します。層ごとに文脈を加え、利用者向けメッセージと内部ログを分けます。



リクエスト側

- OkとErrを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- matchでの処理を小さな責務へ分割
- ?による伝播をResultとログで表現
- panicとの違いをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	OkとErrに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	matchでの処理の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	?による伝播またはpanicとの違いの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

Result・エラー伝播・?演算子を再現して切り分ける

Result・エラー伝播・?演算子の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: OkとErr
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: matchでの処理
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: ?による伝播
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: panicとの違い

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	OkとErr	OkとErr — 入力と前提を確認し、境界を曖昧にしない。
2	matchでの処理	matchでの処理 — 値がどの順で変換されるかを追跡する。
3	?による伝播	?による伝播 — 失敗を再現し、型またはログから原因を特定する。
4	panicとの違い	panicとの違い — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 予想できる失敗はResultで返し、?で文脈を保ったまま呼び出し元へ伝えます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

設定PORTの未設定、不正数値、範囲外を区別するConfigError enumを作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Debug)]
enum ConfigError { Missing, Invalid, OutOfRange }
fn port(raw: Option<&str>) -> Result<u16, ConfigError> {
    let text = raw.ok_or(ConfigError::Missing)?;
    let value = text.parse::<u32>().map_err(|_| ConfigError::Invalid)?;
    u16::try_from(value).map_err(|_| ConfigError::OutOfRange)
}
fn main() { assert!(matches!(port(Some("70000")), Err(ConfigError::OutOfRange))); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 予想できる失敗はResultで返し、?で文脈を保ったまま呼び出し元へ伝えます。
- ✓ Result・エラー伝播・?演算子とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

HashMap・Iterator・closure・モジュール

UNIT 10 ・ CONCEPT

HashMap・Iterator・closure・モジュール

HashMapはキーで値を引き、Iteratorは要素列を遅延変換し、closureは処理を値として渡します。モジュールは公開境界を作り、pubを必要最小限にします。

この単元の到達点

- HashMapの所有権
- Iteratorアダプタ
- closureの捕捉
- modとpub

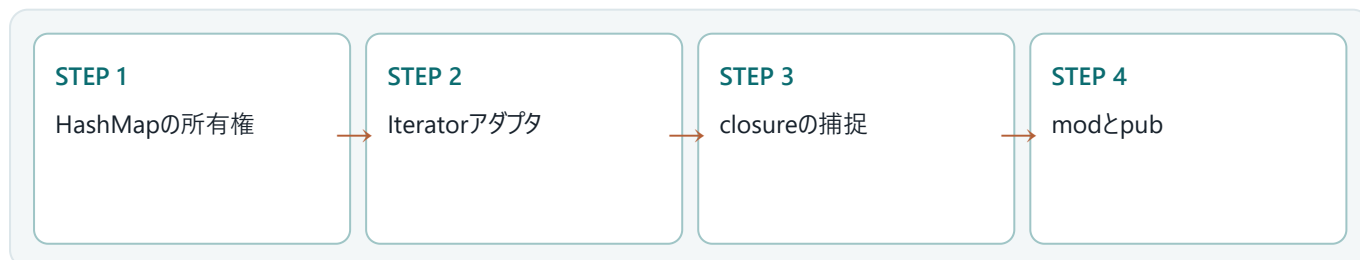
なぜバックエンドで必要か

設定キャッシュや集計にHashMap、DB行からレスポンスへの変換にIterator、層の公開API制御にモジュールを使います。



UNIT 10 ・ MENTAL MODEL

HashMap・Iterator・closure・モジュールを図でつかむ



HashMapはキーで値を引き、Iteratorは要素列を遅延変換し、closureは処理を値として渡します。モジュールは公開境界を作り、pubを必要最小限にします。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	HashMapの所有権	HashMapの所有権 — 入力と前提を確認し、境界を曖昧にしない。
2	Iteratorアダプタ	Iteratorアダプタ — 値がどの順で変換されるかを追跡する。
3	closureの捕捉	closureの捕捉 — 失敗を再現し、型またはログから原因を特定する。
4	modとpub	modとpub — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

HashMap・Iterator・closure・モジュールとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

HashMap・Iterator・closure・モジュールだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use std::collections::HashMap;
fn main() {
    let words = ["rust", "web", "rust", "db"];
    let mut counts = HashMap::new();
    for word in words { *counts.entry(word).or_insert(0) += 1; }
    let mut popular: Vec<_> = counts.into_iter().filter(|(&_, n)| *n >= 2).collect();
    popular.sort_by_key(|(word, _)| *word);
    println!("{popular:?}");
}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use std::collections::HashMap;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let words = ["rust", "web", "rust", "db"];</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let mut counts = HashMap::new();</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>for word in words { *counts.entry(word).or_insert(0) += 1; }</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がHashMapの所有権の条件を満たすか確認する。
- ✓ Iteratorアダプタを実行し、途中の型と状態を追う。
- ✓ closureの捕捉で失敗を成功経路から分離する。
- ✓ modとpubにより、出力と副作用を検証する。

型を声に出す HashMap・Iterator・closure・モジュールに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

["rust", 2]

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Clone)]
struct Note { title: String, done: bool }
fn completed_titles(notes: &[Note]) -> Vec<String> {
    notes.iter().filter(|n| n.done).map(|n| n.title.to_uppercase()).collect()
}
fn main() {
    let notes = [Note { title: "rust".into(), done: true }];
    assert_eq!(completed_titles(&notes), vec!["RUST"]);
}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

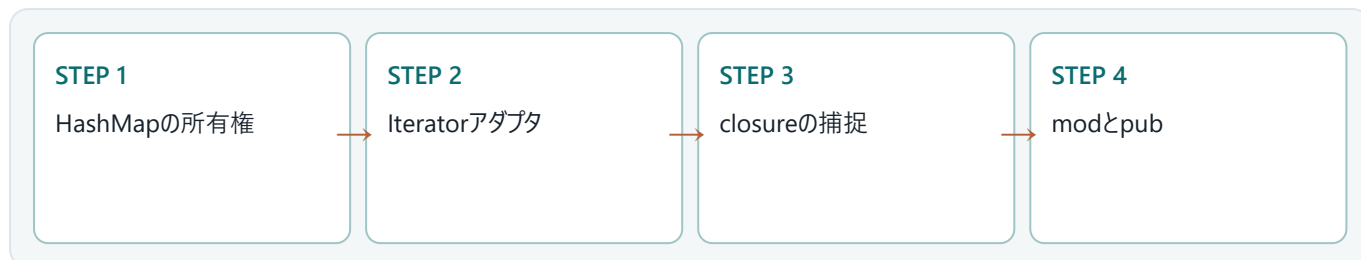
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10・BACKEND APPLICATION

バックエンドのどこで使うか

設定キャッシュや集計にHashMap、DB行からレスポンスへの変換にIterator、層の公開API制御にモジュールを使います。



リクエスト側

- HashMapの所有権を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- Iteratorアダプタを小さな責務へ分割
- closureの捕捉をResultとログで表現
- modとpubをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	HashMapの所有権に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	Iteratorアダプタの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	closureの捕捉またはmodとpubの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

HashMap・Iterator・closure・モジュールを再現して切り分ける

HashMap・Iterator・closure・モジュールの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: HashMapの所有権
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: Iteratorアダプタ
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: closureの捕捉
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: modとpub

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	HashMapの所有権	HashMapの所有権 — 入力と前提を確認し、境界を曖昧にしない。
2	Iteratorアダプタ	Iteratorアダプタ — 値がどの順で変換されるかを追跡する。
3	closureの捕捉	closureの捕捉 — 失敗を再現し、型またはログから原因を特定する。
4	modとpub	modとpub — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 コレクションを所有し、Iteratorで借りながら変換し、モジュールで公開範囲を絞ります。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

Note一覧をdoneだけにfilterし、titleを大文字へmapし、Vec<String>としてcollectしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Clone)]
struct Note { title: String, done: bool }
fn completed_titles(notes: &[Note]) -> Vec<String> {
    notes.iter().filter(|n| n.done).map(|n| n.title.to_uppercase()).collect()
}
fn main() {
    let notes = [Note { title: "rust".into(), done: true }];
    assert_eq!(completed_titles(&notes), vec!["RUST"]);
}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ コレクションを所有し、Iteratorで借りながら変換し、モジュールで公開範囲を絞ります。
- ✓ HashMap・Iterator・closure・モジュールとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

メモリ上で動く認証なしNoteサービスを型安全にする

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	変数・mut・シャドーイング・定数	不変を既定にし、変換段階はシャドーイング 変化する状態だけmutにします。
02	整数・浮動小数・bool・charと変換	数値型は意味と範囲で選び、溢れ・丸め・符号変換を明示します。
03	関数・文・式・戻り値	関数署名で入力と出力を固定し、ブロックの末尾式で値を返します。
04	if・loop・while・for・match	分岐は状態を列挙し、matchの網羅性を設計変更の検知器として使います。
05	String・str・UTF-8	所有が不要なら&strで借り、保存や返却に必要な場所でStringへ変換します。
06	タプル・配列・スライス・Vec	保存形はVec、読み取り専用の関数境界は&[T]を第一候補にします。
07	struct・impl・関連関数・メソッド	structで状態を表し、生成関数で不変条件を守り、implに操作を集めます。
08	enum・Option・パターン分解	存在しない可能性はOption、状態の種類はenumで表し、全候補を処理します。
09	Result・エラー伝播・?演算子	予想できる失敗はResultで返し、?で文脈を保ったまま呼び出し元へ伝えます。
10	HashMap・Iterator・closure・モジュール	コレクションを所有し、Iteratorで借りながら変換し、モジュールで公開範囲を絞ります。

完了条件

- ✓ create・list・toggleが動く
- ✓ 空タイトルと不正IDをResultで拒否する
- ✓ OptionとResultを混同していない
- ✓ 単体テストが正常系と失敗系を覆う

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. 変数・mut・シャドーイング・定数を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. 整数・浮動小数・bool・charと変換を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. 関数・文・式・戻り値を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. if・loop・while・for・matchを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. String・str・UTF-8を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. タプル・配列・スライス・Vecを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. struct・impl・関連関数・メソッドを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. enum・Option・パターン分解を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. Result・エラー伝播・?演算子を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. HashMap・Iterator・closure・モジュールを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. 変数・mut・シャドーイング・定数

変数・mut・シャドーイング・定数とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 不変を既定にし、変換段階はシャドーイング、変化する状態だけmutにします。

02. 整数・浮動小数・bool・charと変換

整数・浮動小数・bool・charと変換とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 数値型は意味と範囲で選び、溢れ・丸め・符号変換を明示します。

03. 関数・文・式・戻り値

関数・文・式・戻り値とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 関数署名で入力と出力を固定し、ブロックの末尾式で値を返します。

04. if・loop・while・for・match

if・loop・while・for・matchとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 分岐は状態を列挙し、matchの網羅性を設計変更の検知器として使います。

05. String・str・UTF-8

String・str・UTF-8とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 所有が不要なら&strで借り、保存や返却に必要な場所でStringへ変換します。

06. タプル・配列・スライス・Vec

タプル・配列・スライス・Vecとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 保存形はVec、読み取り専用の関数境界は&[T]を第一候補にします。

07. struct・impl・関連関数・メソッド

struct・impl・関連関数・メソッドとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: structで状態を表し、生成関数で不変条件を守り、implに操作を集めます。

08. enum・Option・パターン分解

enum・Option・パターン分解とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 存在しない可能性はOption、状態の種類はenumで表し、全候補を処理します。

09. Result・エラー伝播・?演算子

Result・エラー伝播・?演算子とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 予想できる失敗はResultで返し、?で文脈を保ったまま呼び出し元へ伝えます。

10. HashMap・Iterator・closure・モジュール

HashMap・Iterator・closure・モジュールとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: コレクションを所有し、Iteratorで借りながら変換し、モジュールで公開範囲を絞ります。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
変数・mut・シャドーイング・定数	不変を既定にし、変換段階はシャドーイング、変化する状態だけmutにします。	設定値や入力を段階ごとに別の型へ変換し、可変状態は再試行回数など必要な狭い範囲へ閉じ込めます。
整数・浮動小数・bool・charと変換	数値型は意味と範囲で選び、溢れ・丸め・符号変換を明示します。	金額はf64ではなく最小通貨単位の整数で持ち、IDや件数の変換は範囲外をResultで拒否します。
関数・文・式・戻り値	関数署名で入力と出力を固定し、ブロックの末尾式で値を返します。	ハンドラを入力抽出、検証、ユースケース、レスポンス変換へ分けると、それぞれの関数署名が責務の境界になります。
if・loop・while・for・match	分岐は状態を列挙し、matchの網羅性を設計変更の検知器として使います。	HTTP状態、権限、DB結果をmatchで分類し、想定外を明示します。無条件の_で重要な新規状態を隠さない判断も必要です。
String・str・UTF-8	所有が不要なら&strで借り、保存や返却に必要な場所でStringへ変換します。	フォーム入力、URL、DB TEXTはUTF-8文字列です。長さ制限をbytesか文字数のどちらで課すか、DBや外部APIの契約に合わせます。
タプル・配列・スライス・Vec	保存形はVec、読み取り専用の関数境界は&[T]を第一候補にします。	DBから取得した複数行はVec<Model>、関数へ読むだけで渡すときは&[Model]にし、所有権移動と不要な割り当てを避けます。
struct・impl・関連関数・メソッド	structで状態を表し、生成関数で不変条件を守り、implに操作を集めます。	User、Note、Sessionをstructで表し、SQL行、フォームDTO、ドメイン型を分けると、どの値が検証済みかを型で追えます。
enum・Option・パターン分解	存在しない可能性はOption、状態の種類はenumで表し、全候補を処理します。	DBの0件をOption、権限不足を別のエラー-variantとして区別すると、HTTP 404と403を正しく返せます。
Result・エラー伝播・?演算子	予想できる失敗はResultで返し、?で文脈を保ったまま呼び出し元へ伝えます。	HTTP入力、DB、外部APIはすべて失敗します。層ごとに文脈を加え、利用者向けメッセージと内部ログを分けます。
HashMap・Iterator・closure・モジュール	コレクションを所有し、Iteratorで借りながら変換し、モジュールで公開範囲を絞ります。	設定キャッシュや集計にHashMap、DB行からレスポンスへの変換にIterator、層の公開API制御にモジュールを使います。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。